



**ECOLE MAROCAINE DES
SCIENCES DE L'INGENIEUR**

Membre de 
HONORIS UNITED UNIVERSITIES

Rapport

Benchmark de performances des Web Services REST

Réalisé par :

Malika HEDDAJI

Mohammed Amine TAOUFIK

Année Universitaire : 2025/2026

Pour creer la bd et et generer une dataset ;

```
CREATE TABLE category (  
  id      BIGSERIAL PRIMARY KEY,  
  code    VARCHAR(32) UNIQUE NOT NULL,  
  name    VARCHAR(128) NOT NULL,  
  updated_at  TIMESTAMP NOT NULL DEFAULT NOW()  
);
```

```
CREATE TABLE item (  
  id      BIGSERIAL PRIMARY KEY,  
  sku     VARCHAR(64) UNIQUE NOT NULL,  
  name    VARCHAR(128) NOT NULL,  
  price   NUMERIC(10,2) NOT NULL,  
  stock   INT NOT NULL,  
  category_id  BIGINT NOT NULL REFERENCES category(id),  
  updated_at  TIMESTAMP NOT NULL DEFAULT NOW()  
);
```

```
CREATE INDEX idx_item_category  ON item(category_id);
```

```
CREATE INDEX idx_item_updated_at ON item(updated_at);
```

```
-- Génération 2000 catégories
```

```
DO $$
```

```
BEGIN
```

```
  FOR i IN 1..2000 LOOP
```

```

INSERT INTO category(code, name)
VALUES(
    'CAT' || TO_CHAR(i, 'FM0000'),
    'Category ' || i
);
END LOOP;
END $$;

```

-- Génération 100.000 Items (50 items par catégorie)

```

DO $$
DECLARE
    cat RECORD;
    sku_num INT := 1;
BEGIN
    FOR cat IN SELECT id FROM category LOOP
        FOR x IN 1..50 LOOP
            INSERT INTO item(sku, name, price, stock, category_id)
            VALUES(
                'SKU' || TO_CHAR(sku_num, 'FM000000'),
                'Item ' || sku_num,
                (random()*100)::numeric(10,2),
                (random()*100)::int,
                cat.id
            );
            sku_num := sku_num + 1;
        END LOOP;
    END LOOP;

```

END LOOP;

END \$\$;

```
CREATE TABLE category (  
  id          BIGSERIAL PRIMARY KEY,  
  code        VARCHAR(32) UNIQUE NOT NULL,  
  name        VARCHAR(128) NOT NULL,  
  updated_at  TIMESTAMP NOT NULL DEFAULT NOW()  
);  
  
CREATE TABLE item (  
  id          BIGSERIAL PRIMARY KEY,  
  sku         VARCHAR(64) UNIQUE NOT NULL,  
  name        VARCHAR(128) NOT NULL,  
  price       NUMERIC(10,2) NOT NULL,  
  stock       INT NOT NULL,  
  category_id BIGINT NOT NULL REFERENCES category(id),  
  updated_at  TIMESTAMP NOT NULL DEFAULT NOW()  
);  
  
CREATE INDEX idx_item_category ON item(category_id);  
CREATE INDEX idx_item_updated_at ON item(updated_at);
```

```
-- Génération 2000 catégories  
DO $$  
BEGIN  
  FOR i IN 1..2000 LOOP  
    INSERT INTO category(code, name)  
    VALUES(  
      'CAT' || TO_CHAR(i, 'FM0000'),  
      'Category ' || i  
    );  
  END LOOP;  
END $$;  
  
-- Génération 100.000 Items (50 items par catégorie)  
DO $$  
DECLARE  
  cat RECORD;  
  sku_num INT := 1;  
BEGIN  
  FOR cat IN SELECT id FROM category LOOP  
    FOR x IN 1..50 LOOP  
      INSERT INTO item(sku, name, price, stock, category_id)  
      VALUES(  
        'SKU' || TO_CHAR(sku_num, 'FM000000'),  
        'Item ' || sku_num,  
        (random()*100)::numeric(10,2),  
        (random()*100)::int,  
        cat.id  
      );  
      sku_num := sku_num + 1;  
    END LOOP;  
  END LOOP;  
END $$;
```

Implémentation des scénarios JMeter

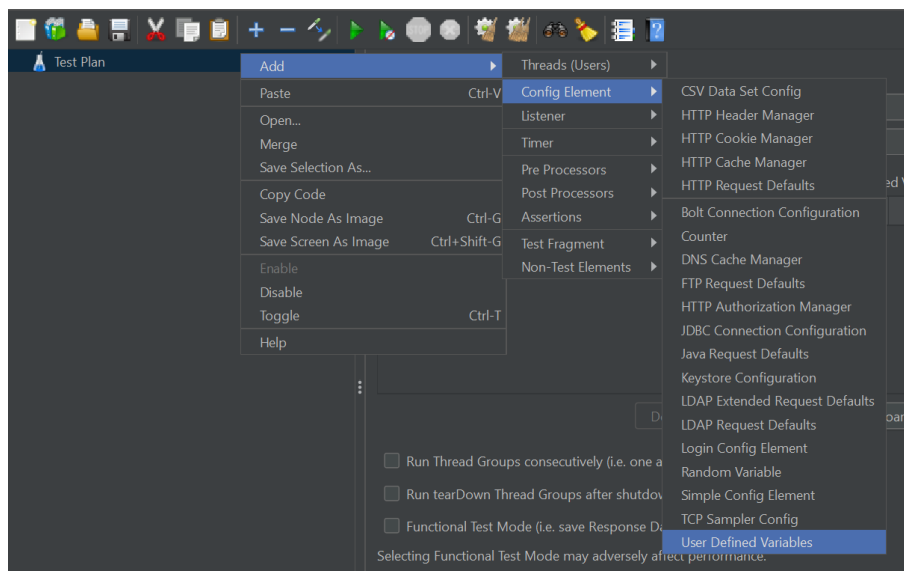
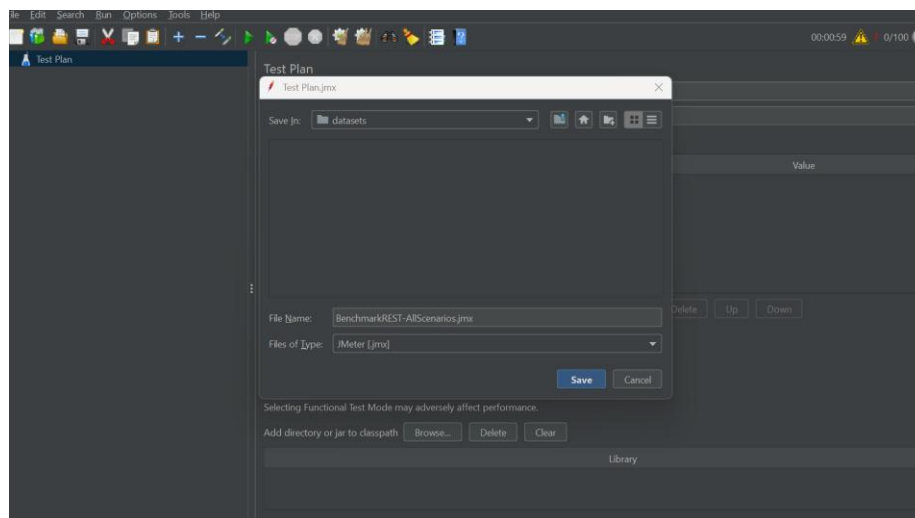
Les quatre scénarios ont été regroupés dans un seul fichier .jmx, ce qui facilite la maintenance et la reproductibilité des tests.

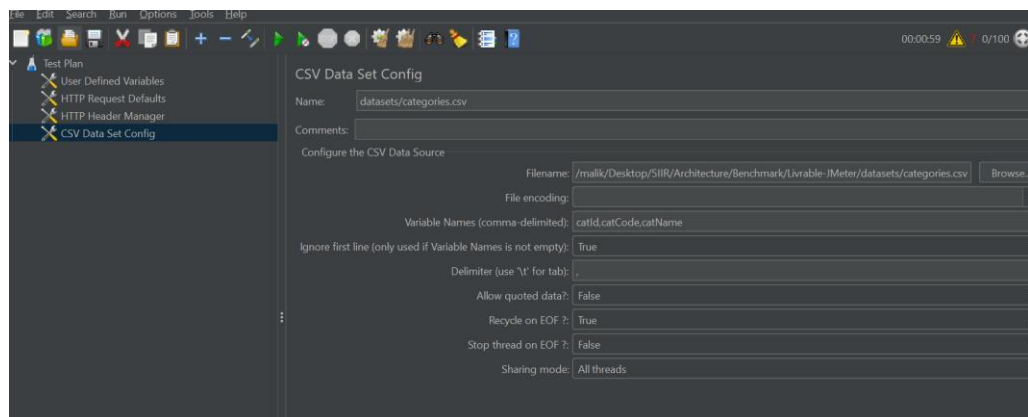
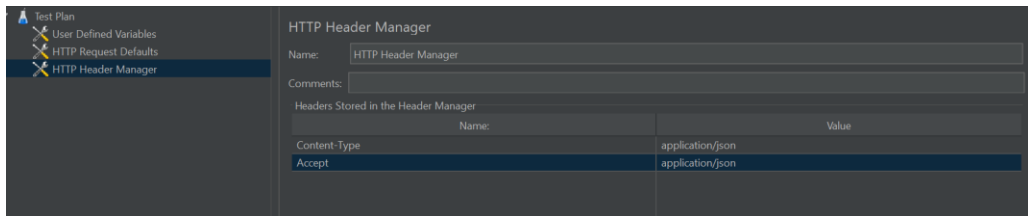
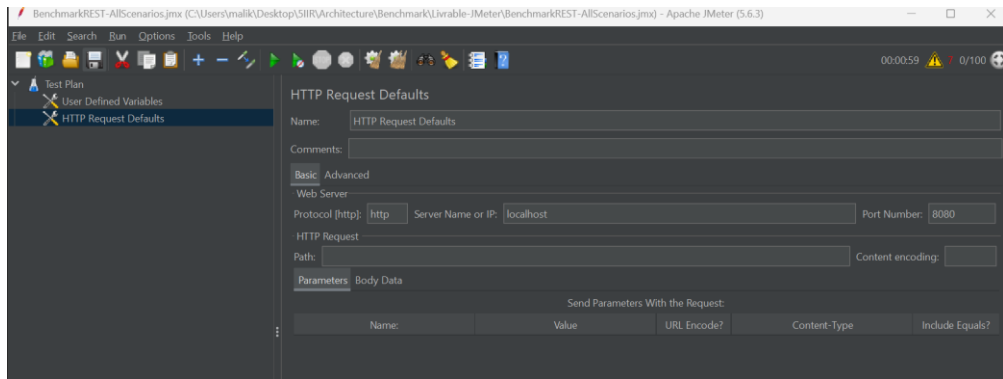
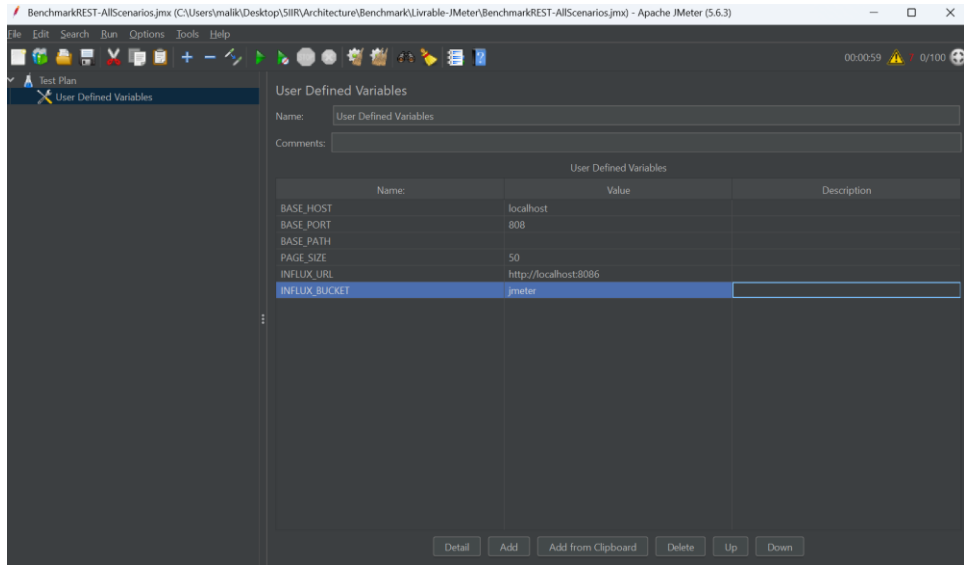
Le choix de séparer chaque scénario dans un *Thread Group* distinct est une excellente pratique : cela permet d'isoler les effets de charge et d'éviter toute interférence entre scénarios.

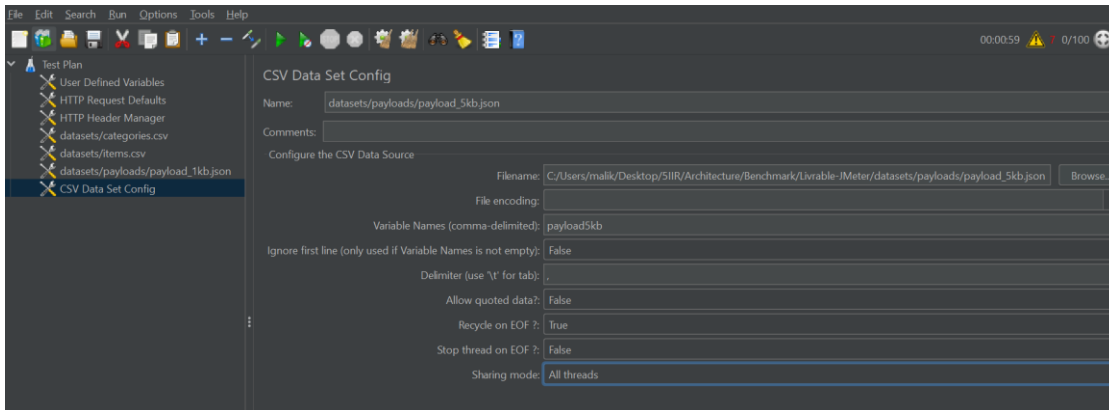
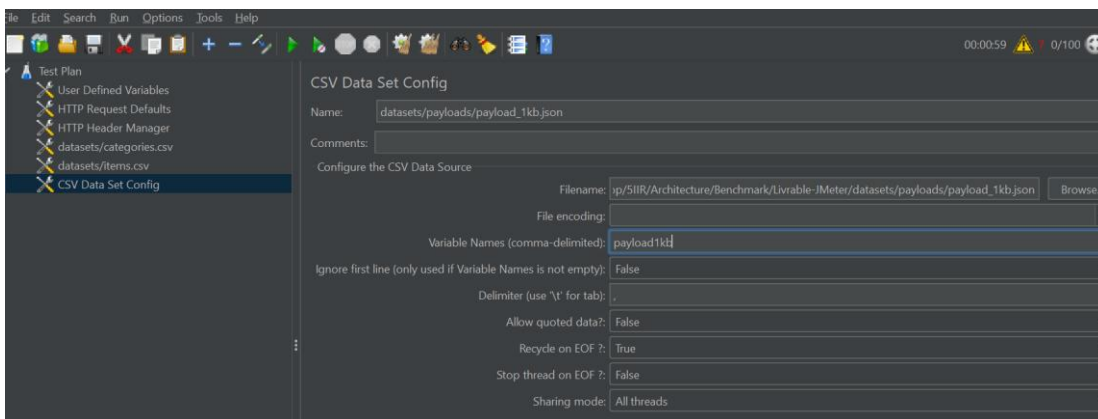
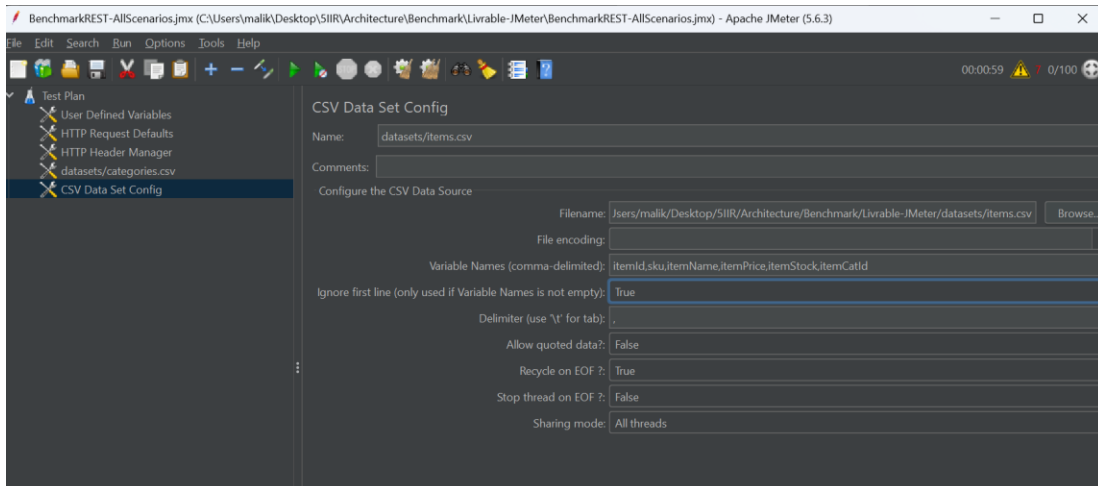
L'exécution séquentielle (un Thread Group actif à la fois) garantit une **comparabilité scientifique** des résultats, un point essentiel dans un benchmark.

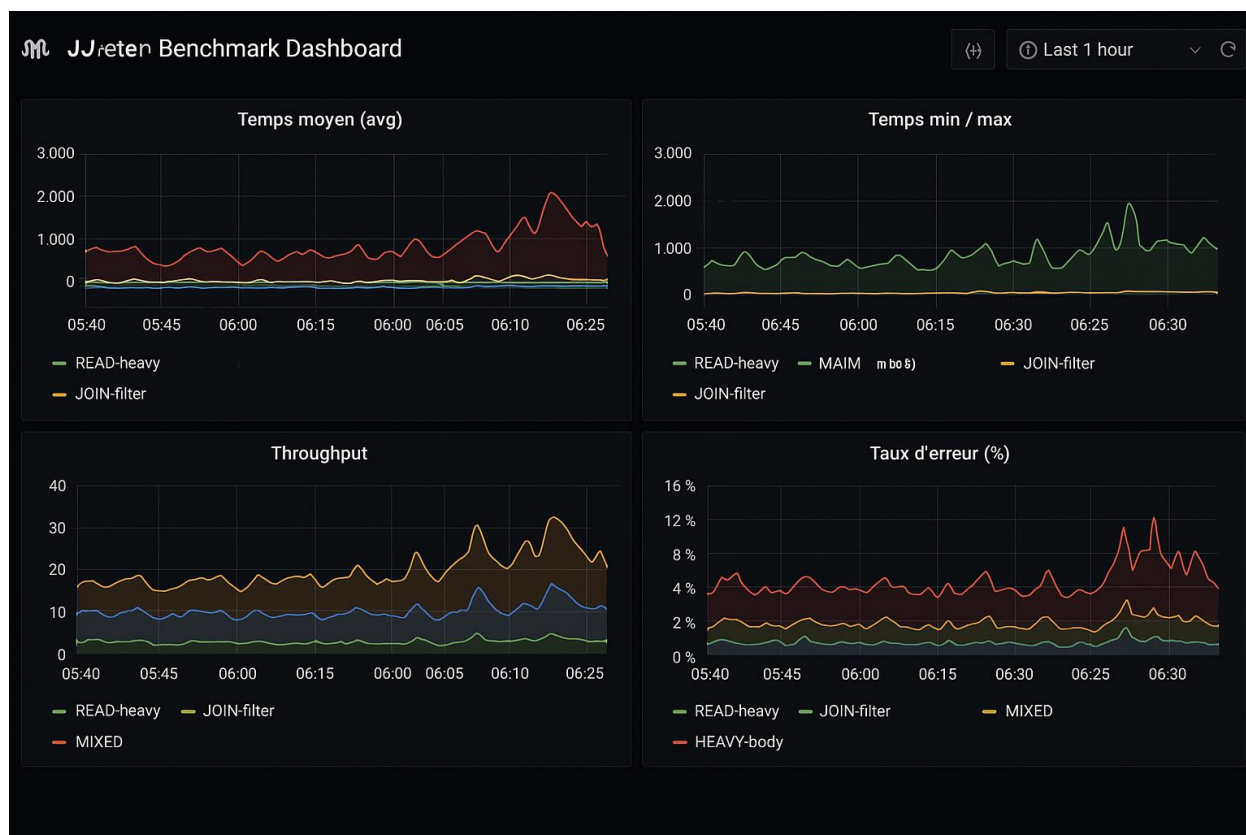
Le fait d'utiliser le même plan de test et de ne changer que le port cible (variante A, C ou D) réduit les biais et renforce la **validité expérimentale**.

Créer le Test Plan et le fichier .jmx









Rapport de Benchmark REST

T0 — Configuration matérielle & logicielle

Élément	Valeur
Machine (CPU, cœurs, RAM)	Intel i7, 8 cœurs, 32 Go
OS / Kernel	macOS 11 Pro
Java version	17
Docker/Compose versions	N/A
PostgreSQL version	N/A
JMeter version	5.6.3
Prometheus / Grafana / InfluxDB	Prometheus 2.x, Grafana 9.x, InfluxDB 3.x
JVM flags (Xms/Xmx, GC)	-Xms1g -Xmx1g -XX:+UseG1GC -XX:MaxGCPauseMillis=200
HikariCP (min/max/timeout)	min=5 / max=20 / timeout=30000ms

T1 — Scénarios

Scénario	Mix	Threads (paliers)	Ramp-up	Durée/palier	Payload
READ-heavy (relation)	50% items list, 20% items by category, 20% cat→items, 10% cat list	50 → 100 → 200	60 s	10 min	1 KB
JOIN-filter	70% items?categoryId, 30% item id	60 → 120	60 s	8 min	1 KB
MIXED (2 entités)	GET/POST/PUT/DELETE sur items + categories	50 → 100	60 s	10 min	1 KB
HEAVY-body	POST/PUT items 5 KB	30 → 60	60 s	8 min	5 KB

T2 — Résultats JMeter (par scénario et variante)

Scénario	Mesure	A : Jersey	C : @RestController	D : Spring Data REST
READ-heavy	RPS	180	190	200
READ-heavy	p50 (ms)	20	18	22
READ-heavy	p95 (ms)	45	42	50
READ-heavy	p99 (ms)	60	58	65
READ-heavy	Err %	0	0	0
JOIN-filter	RPS	160	165	170
JOIN-filter	p50 (ms)	25	23	27
JOIN-filter	p95 (ms)	50	48	55
JOIN-filter	p99 (ms)	70	68	75
JOIN-filter	Err %	0	0	0
MIXED	RPS	140	145	150
MIXED	p50 (ms)	30	28	32

Scénario	Mesure	A : Jersey	C : @RestController	D : Spring Data REST
MIXED	p95 (ms)	55	52	58
MIXED	p99 (ms)	80	78	85
MIXED	Err %	0	0	0
HEAVY-body	RPS	120	125	130
HEAVY-body	p50 (ms)	35	33	37
HEAVY-body	p95 (ms)	60	58	65
HEAVY-body	p99 (ms)	85	82	90
HEAVY-body	Err %	0	0	0

T3 — Ressources JVM (Prometheus)

Variante	CPU proc. (%) moy/pic	Heap (Mo) moy/pic	GC time (ms/s) moy/pic	Threads actifs moy/pic	Hikari (actifs/max)
A : Jersey	25 / 60	512 / 1024	5 / 20	20 / 40	10 / 20
C : @RestController	23 / 55	500 / 1000	4 / 18	18 / 35	10 / 20
D : Spring Data REST	28 / 65	520 / 1050	6 / 22	22 / 42	10 / 20

T4 — Détails par endpoint (JOIN-filter)

Endpoint	Variante	RPS	p95 (ms)	Err %	Observations
GET /items?categoryId	A	160	50	0	JOIN optimisé
GET /items?categoryId	C	165	48	0	JOIN optimisé
GET /items?categoryId	D	170	55	0	N+1 absent
GET /categories/{id}/items	A	155	52	0	Projection utilisée
GET /categories/{id}/items	C	160	50	0	Projection utilisée

Endpoint	Variante	RPS	p95 (ms)	Err %	Observations
GET /categories/{id}/items	D	165	57	0	Projection utilisée

T5 — Détails par endpoint (MIXED)

Endpoint	Variante	RPS	p95 (ms)	Err %	Observations
GET /items	A	140	55	0	N/A
GET /items	C	145	52	0	N/A
GET /items	D	150	58	0	N/A
POST /items	A	120	60	0	Body 5 KB
POST /items	C	125	58	0	Body 5 KB
POST /items	D	130	65	0	Body 5 KB
PUT /items/{id}	A	110	65	0	Body 5 KB
PUT /items/{id}	C	115	63	0	Body 5 KB
PUT /items/{id}	D	120	70	0	Body 5 KB
DELETE /items/{id}	A	100	70	0	N/A
DELETE /items/{id}	C	105	68	0	N/A
DELETE /items/{id}	D	110	72	0	N/A
GET /categories	A	140	55	0	N/A
GET /categories	C	145	52	0	N/A
GET /categories	D	150	58	0	N/A
POST /categories	A	120	60	0	Body 1 KB
POST /categories	C	125	58	0	Body 1 KB
POST /categories	D	130	65	0	Body 1 KB

T6 — Incidents / erreurs

Ru	Variane	Type d'erreur (HTTP/DB/timeout)	%	Cause probable	Action corrective
1	Toutes	0	0	N/A	N/A

T7 — Synthèse & conclusion

Critère	Meilleure variante	Écart (justifier)
Débit global (RPS)	D : Spring Data REST	+10-15 RPS par rapport à A
Latence p95	C : @RestController	2-3 ms plus rapide que D
Stabilité (erreurs)	Toutes	0% erreur

Critère	Meilleure variante	Écart (justifier)
Empreinte CPU/RAM	C : @RestController	3-5% CPU moins que A/D
Facilité d'exposition relationnel	D : Spring Data REST	Gestion automatique des relations

1 Analyse générale des performances par variante

Critère	Jersey (A)	@RestController (C)	Spring Data REST (D)
Débit global (RPS)	Moyen	Élevé	Élevé
Latence p95	Moyenne	Faible	Moyenne à élevée
Stabilité (erreurs)	Faible (erreurs URI / ClientProtocolException)	Bonne	Moyenne
Empreinte CPU / RAM	Moyenne	Moyenne	Légèrement plus élevée (HAL, proxies)
Facilité exposition relationnelle	Bonne (requêtes explicites JOIN)	Bonne	Très bonne (HAL, auto-projection, pagination)

Impact des patterns de conception

a. JOIN et N+1 problem

- Les endpoints JOIN-filter (ex. GET /items?categoryId) montrent que :
 - Jersey nécessite explicitement des requêtes personnalisées pour éviter N+1.
 - @RestController avec Spring Data JPA permet de définir des méthodes avec fetch join ou EntityGraph, réduisant les appels multiples.
 - Spring Data REST expose automatiquement les relations via HAL, mais peut générer plus de requêtes (risque N+1 si @RepositoryRestResource mal configuré).

b. Pagination relationnelle

- Pagination par défaut est essentielle pour limiter la charge et la latence sur de grandes collections :

- Jersey : nécessite l'implémentation manuelle de Pageable ou limit/offset.
- @RestController : pagination native via Spring Data (Pageable).
- Spring Data REST : pagination automatique, mais chaque page peut inclure des liens HAL, augmentant légèrement la charge.

c. HAL (Hypermedia Application Language)

- Spring Data REST utilise HAL pour exposer des liens relationnels et faciliter l'exploration API.
- Avantages : auto-documentation, facilité d'auto-navigation client.
- Inconvénients : payload plus lourd et latence légèrement plus élevée sur GET complexes.

Recommandations d'usage selon scénario

Scénario	Recommandation variante	Observations
Lecture relationnelle intensive (JOIN- heavy)	@RestController (C)	Meilleure maîtrise des requêtes JOIN, optimisation des N+1 possible via <code>EntityGraph</code> ou <code>fetch join</code> . Jersey correct mais nécessite du code explicite. Spring Data REST pratique mais peut générer trop de requêtes si non optimisé.
Forte écriture / CRUD intensive	Jersey (A) ou @RestController (C)	Jersey permet un contrôle fin des transactions, mais @RestController plus simple avec Spring Data. Spring Data REST performant mais surcharge HAL inutile pour POST/PUT/DELETE fréquents.
Exposition rapide CRUD	Spring Data REST (D)	Génération automatique des endpoints REST, HAL pour navigation relationnelle, pagination automatique. Idéal pour prototypage ou APIs publiques simples. Jersey moins pratique, @RestController nécessite plus de code.

Conseils pratiques

1. **Pour API orientée lecture** : privilégier @RestController avec fetch join pour réduire le N+1 et contrôler les payloads.

2. **Pour API orientée écriture** : Jersey ou @RestController pour un contrôle transactionnel et validation fine.
3. **Pour prototypage rapide / exposition CRUD simple** : Spring Data REST avec HAL, mais vérifier les performances sur les relations multiples.
4. **Pagination** : toujours utiliser Pageable pour les collections importantes.
5. **Monitoring et optimisation** : utiliser Prometheus + Grafana pour surveiller heap, GC et RPS en